Database security

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Wykład 10

1

1

The Purpose of Security

- Protecting Data from Theft
- Protecting from Purposeful Damage
- Protecting from Accidental Damage
- Protecting Data from Exposure
- Compliance and Auditing Standards

2

2

Threats to Databases

- **Loss of integrity**. Protecting database from improper modification: creating, inserting, updating, changing and deleting data.
- **Loss of availability**. Loss of availability occurs when the user or program who/which has a legitimate right cannot access these objects.
- **Loss of confidentiality**. Protection of data from unauthorized disclosure of confidential information, for ex. violation of the GDPR.

3

3

Types of database security mechanisms

- **Discretionary Access Control** (DAC) - used to grant privileges to users, including the capability to access specific data files, records, or fields in a specified mode (such as read, insert, delete, or update).
- **Role-Based Access Control** (RBAC) - enforces policies and privileges based on the concept of organizational roles.
- **Mandatory Access Control** (MAC) - used to enforce multilevel security by classifying the data and users into various security classes (or levels) and then implementing the appropriate security policy of the organization.

4

4

Security factors

- **Data sensitivity**. The value of the data itself may be so revealing or confidential that it becomes sensitive— salary or who has HIV/AIDS.
- **Access acceptability**. Data should only be revealed to authorized users.
- **Authenticity assurance**. Before granting access, certain external characteristics about the user may also be considered. For example, a user may only be permitted access during working hours. The system may track previous queries to ensure that a combination of queries does not reveal sensitive data. The latter is particularly relevant to statistical database queries

5

5

Database administrator (DBA)

1. **Account creation**. This action creates a new account and password for a user or a group of users to enable access to the DBMS.
2. **Privilege granting**. This action permits the DBA to grant certain privileges to certain accounts.
3. **Privilege revocation**. This action permits the DBA to revoke (cancel) certain privileges that were previously given to certain accounts.
4. **Security level assignment**. This action consists of assigning user accounts to the appropriate security clearance level.

6

6

Access control

```
CREATE USER user IDENTIFIED BY 'password';
```

```
CREATE USER user IDENTIFIED BY 'password'
     DEFAULT ROLE administrator, developer
     WITH PASSWORD EXPIRE 120 DAY
          MAX_QUERIES_PER_HOUR 500
          MAX_UPDATES_PER_HOUR 100
          FAILED_LOGIN_ATTEMPTS 3
          PASSWORD_LOCK_TIME 2;
```

```
DROP USER user;
```

7

7

Discretionary privileges

- The account level. At this level, the DBA specifies the particular privileges that each account holds independently of the relations in the database:
  - `CREATE SCHEMA/DATABASE`
  - `CREATE TABLE`
  - `CREATE VIEW`

The user has full privileges to the objects created by him/her.

- The relation (or table) level. At this level, the DBA can control the privilege to access each individual relation or view in the database:
  - `ALL, SELECT, INSERT, UPDATE, DELETE, CREATE, CREATE VIEW, DROP, EXECUTE, ALTER, ALTER ROUTINE,…`

8

8

Discretionary privileges

```
GRANT SELECT, INSERT, UPDATE
ON some_table TO user
WITH GRANT OPTION;
```

```
REVOKE INSERT, UPDATE
ON some_table FROM user;
```

```
SHOW GRANTS FOR user;
```

9

9

Discretionary privileges

DBA:

- `CREATE SCHEMA EXAMPLE; GRANT ALL ON EXAMPLE TO A1 WITH GRANT OPTION;`

USER A1:

- `CREATE TABLE EMPLOYEE (Name NCHAR(50), Depart NCHAR(50), Salary FLOAT);`

10

10

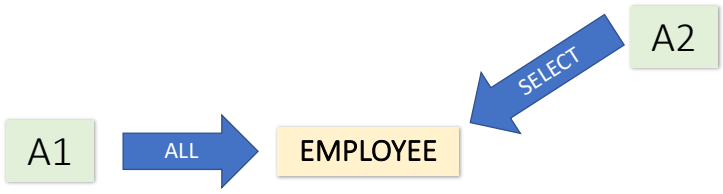
Discretionary privileges

DBA:

- `CREATE SCHEMA EXAMPLE; GRANT ALL ON EXAMPLE TO A1 WITH GRANT OPTION;`

USER A1:

- `CREATE TABLE EMPLOYEE (Name NCHAR(50), Depart NCHAR(50), Salary FLOAT);`
- `GRANT SELECT ON EMPLOYEE TO A2 WITH GRANT OPTION;`



11

11

Discretionary privileges

DBA:

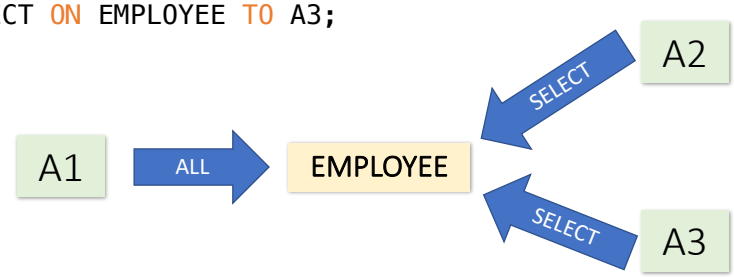
- `CREATE SCHEMA EXAMPLE; GRANT ALL ON EXAMPLE TO A1 WITH GRANT OPTION;`

USER A1:

- `CREATE TABLE EMPLOYEE (Name NCHAR(50), Depart NCHAR(50), Salary FLOAT);`
- `GRANT SELECT ON EMPLOYEE TO A2 WITH GRANT OPTION;`

USER A2:

- `GRANT SELECT ON EMPLOYEE TO A3;`



12

12

Discretionary privileges

DBA:

- `CREATE SCHEMA EXAMPLE; GRANT ALL ON EXAMPLE TO A1 WITH GRANT OPTION;`

USER A1:

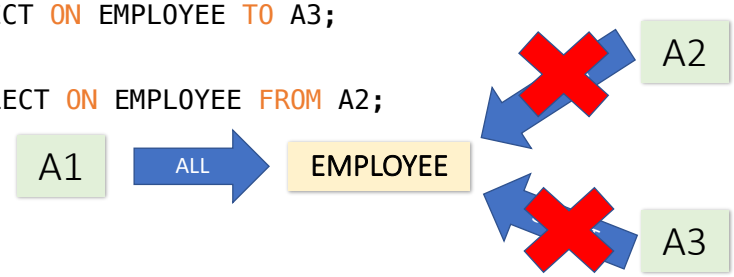
- `CREATE TABLE EMPLOYEE (Name NCHAR(50), Depart NCHAR(50), Salary FLOAT);`
- `GRANT SELECT ON EMPLOYEE TO A2 WITH GRANT OPTION;`

USER A2:

- `GRANT SELECT ON EMPLOYEE TO A3;`

USER A1:

- `REVOKE SELECT ON EMPLOYEE FROM A2;`



13

13

Discretionary privileges

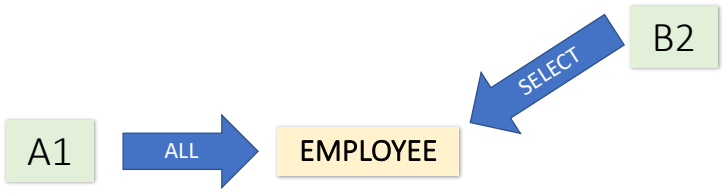
14

14

Discretionary privileges

USER A1:

- `GRANT SELECT ON EMPLOYEE TO B2 WITH GRANT OPTION;`



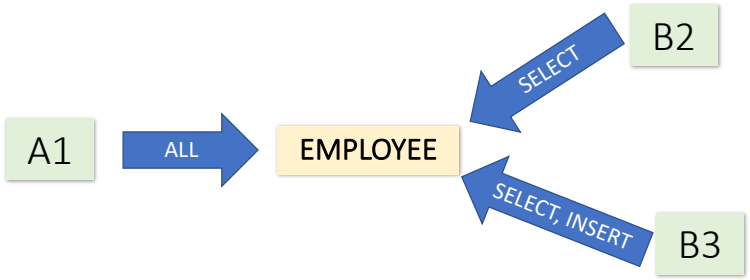
15

15

Discretionary privileges

USER A1:

- `GRANT SELECT ON EMPLOYEE TO B2 WITH GRANT OPTION;`
- `GRANT SELECT, INSERT ON EMPLOYEE TO B3 WITH GRANT OPTION;`



16

16

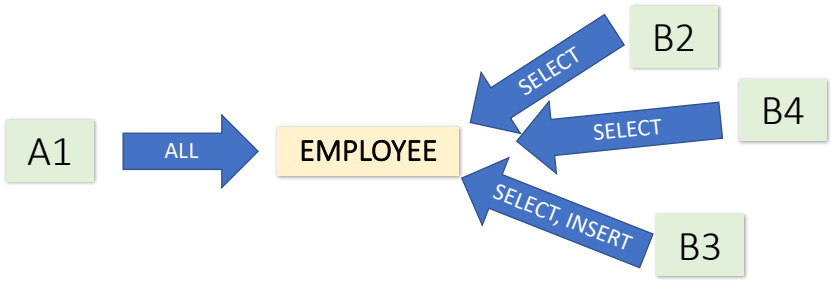
Discretionary privileges

USER A1:

- `GRANT SELECT ON EMPLOYEE TO B2 WITH GRANT OPTION;`
- `GRANT SELECT, INSERT ON EMPLOYEE TO B3 WITH GRANT OPTION;`

USER B2:

- `GRANT SELECT ON EMPLOYEE TO B4;`



17

17

Discretionary privileges

USER A1:

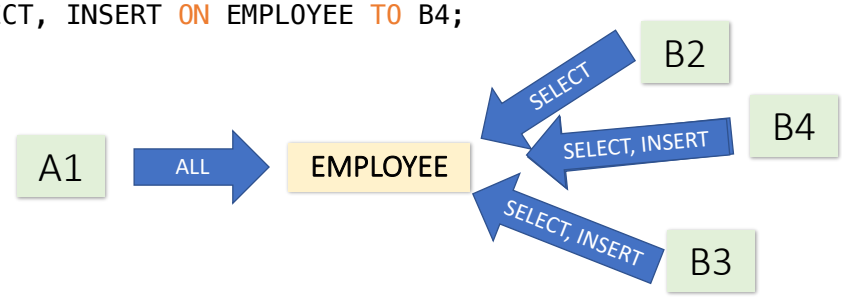
- `GRANT SELECT ON EMPLOYEE TO B2 WITH GRANT OPTION;`
- `GRANT SELECT, INSERT ON EMPLOYEE TO B3 WITH GRANT OPTION;`

USER B2:

- `GRANT SELECT ON EMPLOYEE TO B4;`

USER B3:

- `GRANT SELECT, INSERT ON EMPLOYEE TO B4;`



18

18

Discretionary privileges

USER A1:

- `GRANT SELECT ON EMPLOYEE TO B2 WITH GRANT OPTION;`
- `GRANT SELECT, INSERT ON EMPLOYEE TO B3 WITH GRANT OPTION;`

USER B2:

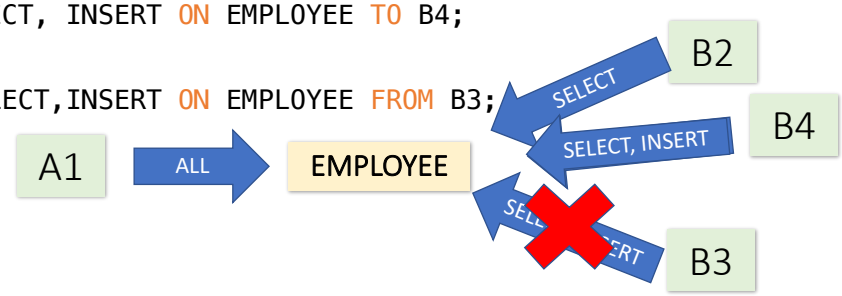
- `GRANT SELECT ON EMPLOYEE TO B4;`

USER B3:

- `GRANT SELECT, INSERT ON EMPLOYEE TO B4;`

USER A1:

- `REVOKE SELECT,INSERT ON EMPLOYEE FROM B3;`



19

19

Discretionary pr. – procedures & functions

```
GRANT CREATE ROUTINE
ON some_db.* TO user;
```

```
GRANT EXECUTE, ALTER ROUTINE, DROP
ON PROCEDURE some_procedure TO user
WITH GRANT OPTION;
```

20

20

Discretionary privileges – VIEW

EmpID	Name	Depart	Salary
E0045	Smith	sales	55 000
E3542	Mike	logistics	51 000
E1111	Smith	manufacturing	39 000
E9999	Mary	manufacturing	45 000

```
CRETE VIEW view_employees_manuf AS
   SELECT * FROM EMPLOYEES
   WHERE Depart = 'manufacturing';
```

```
CRETE VIEW view_employees_manuf_limited AS
   SELECT EmpID, Name FROM EMPLOYEES
   WHERE Depart = 'manufacturing';
```

21

21

Discretionary privileges

```
CRETE VIEW view_employees_manuf AS
SELECT Name FROM EMPLOYEES
WHERE Depart = 'manufacturing';
```

- Grant privilege to all data stored in the attribute (all records):

```
GRANT SELECT (Name)
ON EMPLOYEES TO A5;
```

- Grant privilege a limited data (selected records)

```
GRANT INSERT, UPDATE, SELECT
ON view_employees_manuf TO A4;
```

22

22

Role-Based Access Control

```
CREATE ROLE developer, data_analyst;
```

```
GRANT SELECT, INSERT, UPDATE ON table_name TO developer;
```

```
GRANT developer TO John_Doe;
```

```
REVOKE INSERT, UPDATE ON table_name FROM developer;
```

```
REVOKE developer FROM John_Doe;
```

```
DROP ROLE developer;
```

23

23

The Principle of Least Privilege (POLP)

- Required permissions for a task shall only grant access to the needed information or resources that a task requires.
- When permissions are granted, we shall grant the least privileges possible.
- Avoid unused permissions.
- Audit permissions in a periodical manner.

- SQL isn't perfect:
  - DBA: GRANT SELECT, INSER, UPDATE, ALTER ON some_table TO user;
  - user: DROP TABLE some_table;
  - user: ALTER TABLE some_table DROP COLUMN column1;

ERROR

SUCCESS

24

24

Separation of Duty (SOD)

- No user should be given enough privileges to misuse the system on their own.
- For example, the person authorizing a paycheck should not also be the one who can prepare them.
- Separation of duties can be enforced either:
  - **statically** – by defining conflicting roles, i.e., roles that the same user cannot execute,
  - **dynamically** – by enforcing the control at access time.
  - An example of dynamic separation of duty is the two-person rule. The first user to execute a two-person operation can be any authorized user, whereas the second user can be any authorized user different from the first.

Hu, V. C., Kuhn, R., & Yaga, D. (2017). Verification and test methods for access control policiesmodels. https://doi.org/10.6028/nist.sp.800-192

25

25

Mandatory Access Control (MAC)

- MAC model is a security model that enforces a strict set of rules to determine who has access to certain resources.
- Each **data object is labelled** (for ex. public, confidential, and sensitive) with a certain classification level.
- It compares the security label of the user requesting access against the security label of the resource.
- MAC model determines access to resources using a hierarchical structure.
- Access is denied if the user's security label is lower than the resource's security label.
- **MAC is often used in sensitive environments such as military and government organizations,** where strict control over access to resources is necessary.

26

26

Mandatory Access Control (MAC)

- Enforce label checks on SELECT, INSERT, UPDATE, and DELETE.
- Block cross-label writes and unauthorized changes to reduce insider risk and human error.

27

27

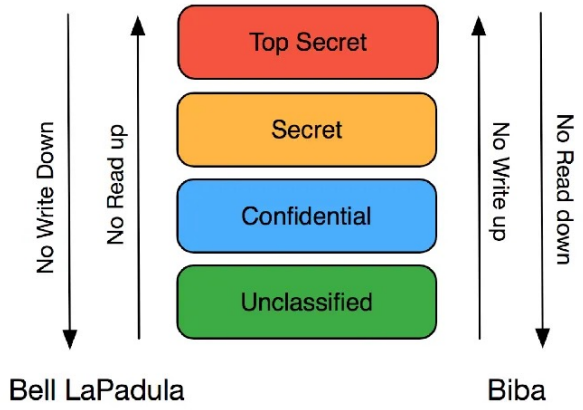
Mandatory Access Control

- Requires special version of OS:
  - Security-Enhanced Linux (SELinux)
  - Since Windows Server 2008 – Mandatory Integrity Control
  - MacOS, iOS
  - Android 5.0+ (based on SELinux)
- In relational dababases:
  - SEPostgreSQL
  - Oracle Label Security
  - IBM DB2 LBAC (Label-Based Access Control)

28

28

Bell-LaPadula and Biba model



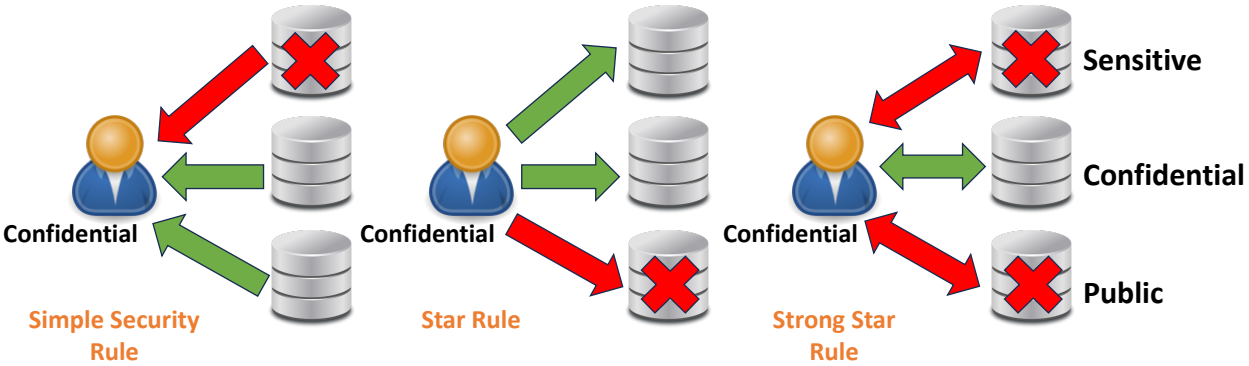
- The **Bell-LaPadula** model protects data confidentiality within a system. Following the "no read up, no write down" principle, users can read data only at their security level or below and write data only at their level or above. This model is often used in military and government systems where preventing information leakage is crucial.
- The **Biba model** focuses on data integrity with its core principle: "no write up, no read down." Users can only write data at their integrity level or below and read data at their integrity level or above. This system prevents lower-integrity users from corrupting higher-integrity data and ensures decisions are based on reliable data sources.

29

29

Bell-LaPadula model (1976)

1. **Simple Security Rule**: No read UP
2. **Star Rule**: No write DOWN
3. **Strong Star Rule:** No read, write UP DOWN

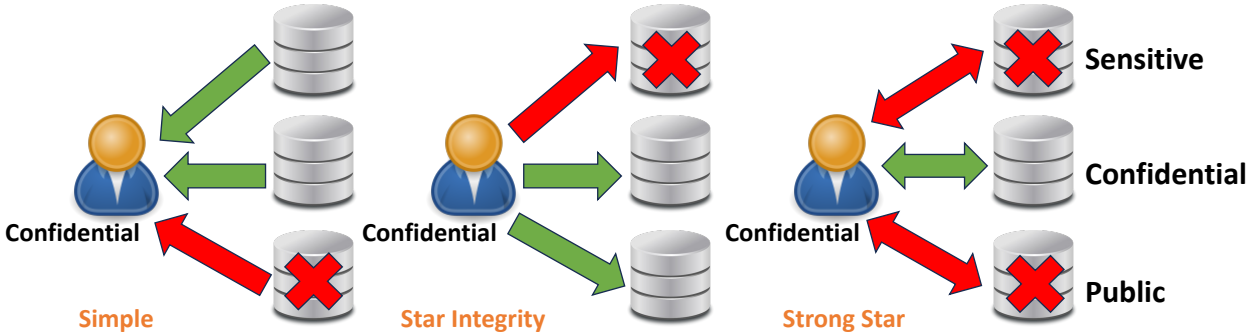


30

30

Biba model (1977)

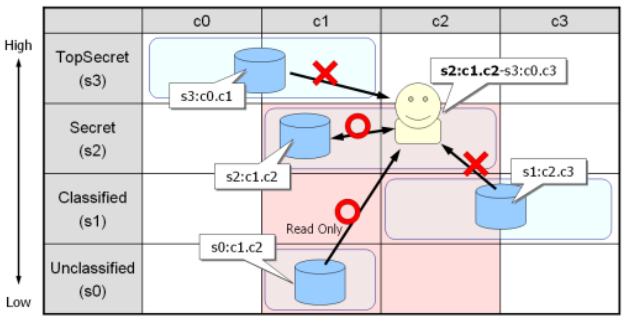
1. **Simple Integrity Rule**: No read DOWN
2. **Star Integrity Rule**: No write UP
3. **Strong Star Integrity Rule:** No read, write UP DOWN



31

31

MAC in SEPostgreSQL + SELinux (Bell-LaPadula)



The following rules are requirements by MLS (Multi Level Security) policy:

- When a process tries to read a resource, its sensitivity has to be equal or lower than the sensitivity of the process.
- When a process tries to read a resource, its categories have to be equal or dominated by the categories of the process.
- When a process tries to write a resource, its range is has to be equal to the lower-range of the process.

32

32

Row-Level Security

ID	Name	Salary	JobPerformance	Security Context (Hidden Label)
101	Smith	40 000	Fair	**secret**
102	Jones	120 000	Excellent	**top secret**
103	Brown	80 000	Good	**confidential**

- ✓ PostgreSQL
- ✓ Microsoft SQL Server
- ✓ Oracle
- ✓ IBM DB2

Secret

```
SELECT * FROM EMPLOYEES;
```

ID	Name	Salary	JobPerformance
101	Smith	40 000	Fair
103	Brown	80 000	Good

Confidential

```
SELECT * FROM EMPLOYEES;
```

ID	Name	Salary	JobPerformance
103	Brown	80 000	Good

top secret (TS) > secret (S) > confidential (C) > unclassified (U)

33

33

Element-Level Security

The original table:

ID	Name	Salary	JobPerformance
101 U	Smith U	40 000 C	Fair S
102 TS	Jones TS	120 000 TS	Excellent TS
103 C	Brown C	80 000 S	Good C

- ✓ Oracle
- ✓ IBM DB2

Confidential

```
SELECT * FROM EMPLOYEES;
```

ID	Name	Salary	JobPerformance
101	Smith	40 000	NULL
103	Brown	NULL	Good

Unclassified

```
SELECT * FROM EMPLOYEES;
```

ID	Name	Salary	JobPerformance
101	Smith	NULL	NULL

top secret (TS) > secret (S) > confidential (C) > unclassified (U)

34

34

Polyinstantiation (with Element-Level Security)

ID	Name	Salary	JobPerformance
101 U	Smith U	40 000 C	Fair S
102 C	Brown C	80 000 S	Good C
103 TS	Jones TS	75 000 TS	Excellent TS

Confidential

```
UPDATE EMPLOYEES
SET JobPerformance = 'Excellent'
WHERE Name = 'Smith';
```

ID	Name	Salary	JobPerformance
101	Smith U	40 000 C	Fair S
101	Smith U	40 000 C	Excellent C
102	Brown C	80 000 S	Good C
103 TS	Jones TS	75 000 TS	Excellent TS

top secret (TS) > secret (S) > confidential (C) > unclassified (U)

35

35

Encrypted Connections

- Encrypted connections between clients and the server using the TLS (Transport Layer Security) protocol.
- TLS uses encryption algorithms to ensure that data received over a public network can be trusted. It has mechanisms to detect data change, loss, or replay. TLS also incorporates algorithms that provide identity verification using the X.509 standard. X.509 makes it possible to identify someone on the Internet.

```
CREATE/ALTER USER user REQUIRE SSL;
```

```
CREATE/ALTER USER user REQUIRE X509;
```

```
SET PERSIST require_secure_transport=ON;
```

Set requirement that clients connect using encrypted connections

36

36

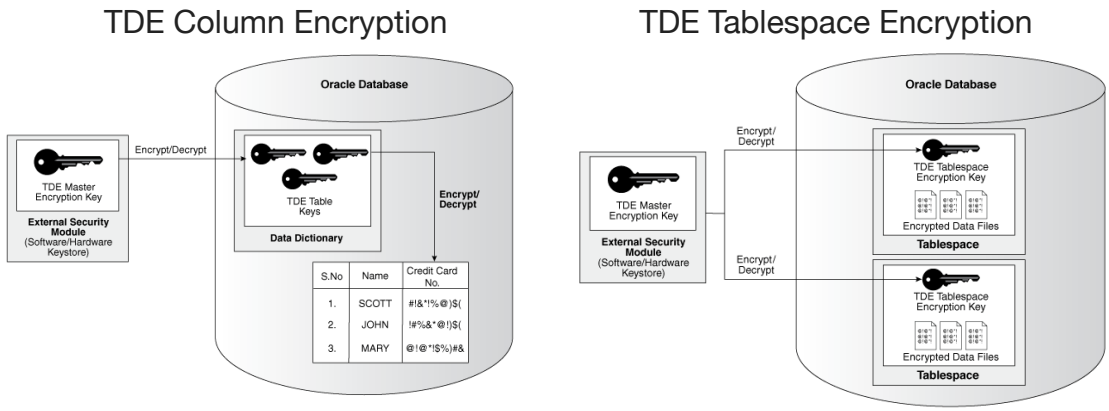
Database encryption levels

- **Cell-Level**: each individual cell of data has its own unique password. Managing the many associated keys requires careful organization.
- **Column-Level**: This is the most commonly known encryption level and is typically included by database vendors. It's possible to implement row-level encryption in which each row of data is encrypted with its own key.
- **Tablespace-Level**: This method doesn't have as much of an impact on performance.

37

37

Transparent Data Encryption (TDE)



źródło: oracle.com

38

38

Transparent Data Encryption (TDE)

- TDE is the encryption of an entire database (within DBMS), including backups (for ex. log).
- “transparent” because it is invisible to users and applications and is easily used without making any application-level changes (doesn’t require code modification of the database or application).
- It is decrypted for authorized users or applications when in use but remains protected at rest.
- Even if the physical media is compromised or the files stolen, the data remains unreadable—only authorized users can successfully read the data.

39

39

TDE in SQL Server

```
USE master;

CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<UseStrongPasswordHere>’;

CREATE CERTIFICATE MyServerCert WITH SUBJECT = 'My DEK Certificate’;

USE AdventureWorks2012;

CREATE DATABASE ENCRYPTION KEY WITH ALGORITHM = AES_256 ENCRYPTION
BY SERVER CERTIFICATE MyServerCert;

ALTER DATABASE AdventureWorks2012 SET ENCRYPTION ON;
```

40

40

Audit Log File

The audit enables DBMS to produce a log file containing an audit record of server activity.

The log contents include when clients connect and disconnect, and what actions they perform while connected, such as which databases and tables they access.

```
<AUDIT_RECORD>
    <TIMESTAMP>2025-05-20T14:09:38 UTC</TIMESTAMP>
    <RECORD_ID>6_2025-05-20T14:06:33</RECORD_ID>
    <NAME>Query</NAME>
    <CONNECTION_ID>5</CONNECTION_ID>
    <STATUS>0</STATUS>
    <STATUS_CODE>0</STATUS_CODE>
    <USER>root[root] @ localhost [127.0.0.1]</USER>
    <OS_LOGIN/>
    <HOST>localhost</HOST>
    <IP>127.0.0.1</IP>
    <COMMAND_CLASS>drop_table</COMMAND_CLASS>
     <SQLTEXT>DROP TABLE IF EXISTS tableA</SQLTEXT>
</AUDIT_RECORD>
```

41

41

Database attacks

- Inference attack
- SQL injection
- Denial of Service
- Cross-Site Scripting
- Weak Authentication
- Excessive privileges

42

42

Inference attack

learning a protected fact without directly accessing it

ID	Name	Salary	JobPerformance	Security Context (Hidden Label)
101	Smith	40 000	Fair	**secret**
102	Jones	120 000	Excellent	**top secret**
103	Brown	80 000	Good	**confidential**

```
SELECT * FROM EMPLOYEES;
```

ID	Name	Salary	JobPerformance
101	Smith	40 000	Fair
103	Brown	80 000	Good

Secret

```
SELECT AVG(Salary), COUNT(*)
FROM EMPLOYEES;
```

AVG(Salary)	COUNT(*)
80 000	3

Based on all records

AVG(Salary)	COUNT(*)
60 000	2

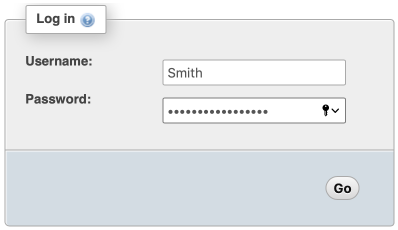
Mitigation: Based on MAC

3x80 000 – 40 000 – 80 000 = 120 000

43

43

SQL injection



- Username = `Smith`
- Password = `fM6d%fr`

```
SELECT * FROM Users WHERE Name = 'Smith' AND password = 'fM6d%fr';
```

- Username = `'OR'' = '`
- Password = `'OR'' = '`

```
SELECT * FROM Users WHERE Name = '' OR '' = ''AND password = '' OR'' = '';
```

`'' = '' is always TRUE`

44

44

SQL injection

- …/viewProduct.php?id=1; DROP TABLE products

```
SELECT id, name, description FROM products WHERE id = 1;

DROP TABLE products;
```

- …/viewProduct.php?id=1; UPDATE users SET email='abc@abc.yy' WHERE user='admin';

```
SELECT id, name, description FROM products WHERE id = 1;

UPDATE users SET email = 'abc@abc.yy' WHERE user ='admin';
```

45

45

SQL injection - risks

- **Database fingerprinting**. The attacker can determine the type of database being used in the backend so that he can use database-specific attacks that correspond to weaknesses in a particular DBMS.
- **Bypassing authentication**. The attacker can gain access to the database as an authorized user and perform all the desired tasks.
- **Identifying injectable parameters**. The attacker gathers important information about the type and structure of the back-end database of a Web application. The default error page returned by application servers is often overly descriptive.
- **Executing remote commands**. This provides attackers with a tool to execute arbitrary commands on the database. For example, a remote user can execute stored database procedures and functions from a remote SQL interactive interface.
- **Performing privilege escalation**. This type of attack takes advantage of logical flaws within the database to upgrade the access level.

46

46

SQL injection - mitigation

- Treat all user input as untrusted. Any user input that is used in an SQL query introduces a risk of an SQL Injection.
- Remove potential malicious code elements such as single quotes.
- The application code should never use the input directly. Use prepared statements, parameterized queries or stored procedures instead whenever possible.
- Turn off the visibility of database errors on production sites. Database errors can be used with SQL Injection to gain information about your database.
- Use appropriate privileges: don't connect to your database using an account with admin-level privileges. Using a limited access account is far safer and can limit what a hacker is able to do.

47

47

Denial of Service (DoS)

Attackers' tactics:

- Building large shopping carts
- Searches with no input or broad input
- API calls that return slowly can tell an abuser that a query might not be optimized or indexed, and thus be a target for repetitive calls
- Adding UNIONs to inputs in forms that can cause huge numbers of joins and scans (this is a SQL injection technique also)
- Sorting large result sets
- Creating edge cases such as a huge number of posts in a forum application, or a huge number of friends in a social application

48

48

Denial of Service (DoS)

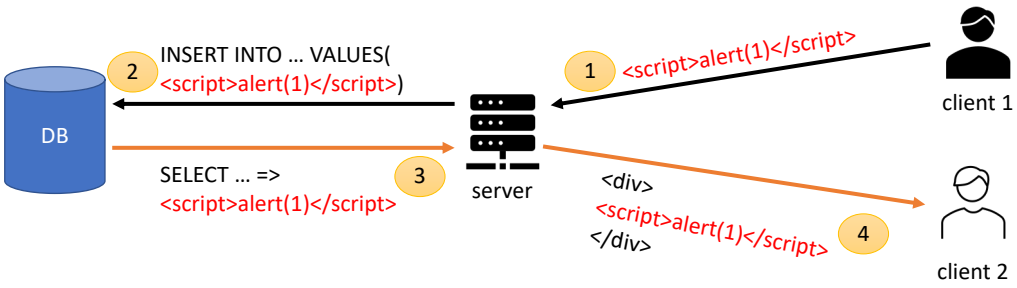
Mitigations:

- **Client-side throttling** – time delay between submitting a request and resubmitting.
- **Quality of service** – classifying traffic going to the application based on criticality, marking expensive queries, such as search.
- **Degrade the results** – for normal loads, a full execution path will be fine, but reduced the number of rows scanned or shards queried during heavy loads.
- **Query killers** – killing long-running queries or putting performance profiles at the database layer to reduce the number of resources a query can consume.

49

49

Cross-Site Scripting (XSS)



For XSS attacks to succeed, an attacker must insert and execute malicious content on a webpage. Each variable in a web application needs to be protected.

50

50

Cross-Site Scripting (XSS)

- Prevent your website from being used to launch XSS attacks.
  - The simplest technique is to disallow any tags in text input by users.
- Protect your website from XSS attacks launched from other sites.
  - Instead of using only the cookie to identify a session, the session could also be restricted to the IP address from which it was originally authenticated.

51

51

Weak Authentication

- Brute forcing of the user interface and insecure storage of the database credentials used by an application.
- Mitigations:
  1. Implement account lockout after a set number of invalid attempts.
  2. Use password blacklisting to prevent users from choosing common passwords.
  3. Don't require users to regularly change their passwords as this encourages easy to remember (and hence guess) passwords.
  4. Consider implementing multi-factor authentication so that an attacker needs more than knowledge of a username and password to access data illegally.
  5. Don't store user passwords in the clear text.

```
CREATE USER user
      PASSWORD HISTORY 5
      PASSWORD REUSE INTERVAL 365 DAY;
```

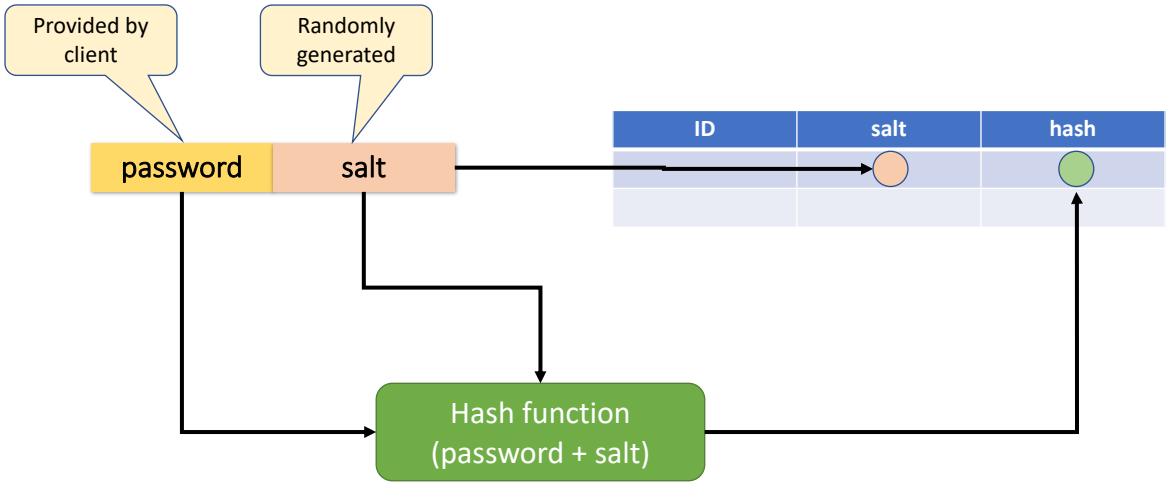
Require a minimum of 5 password changes before permitting reuse

Require a minimum of 365 days elapsed before permitting reuse

52

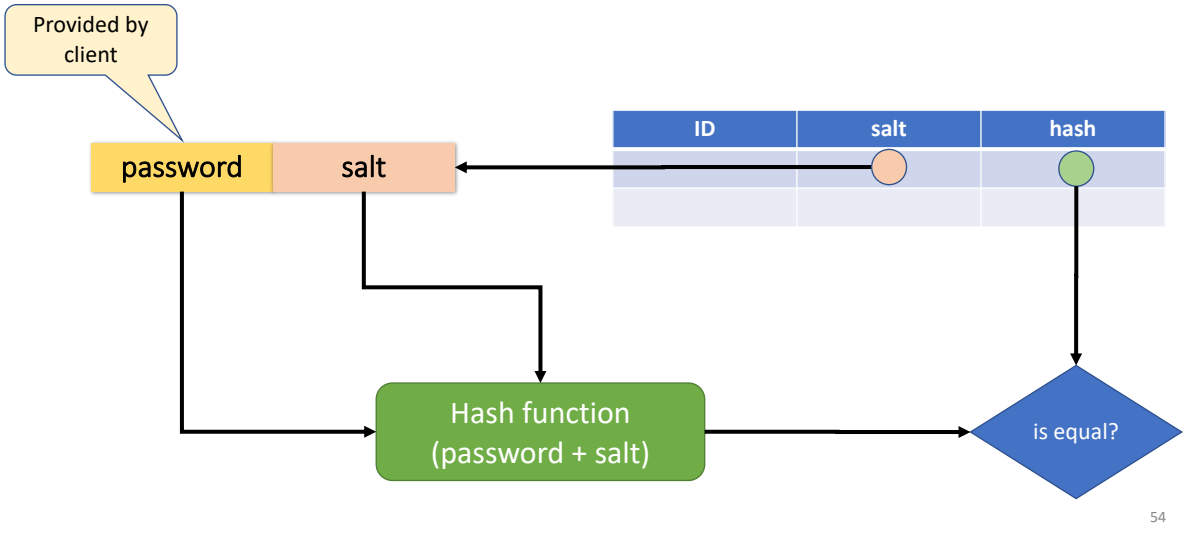
52

Secure storing passwords in DB



53

Password validation in DB



54

Excessive privileges

- If users hold privileges that exceed the requirements of their job function, these privileges may be abused by the individual or an attacker who compromises their account. When people move roles, they may be given the new privileges they need without those they no longer require being removed.
- Mitigations:
  1. Role-based access controls within the application accurately map required access permissions to job functions.
  2. Procedures that ensure that when staff changes roles, their permissions are updated to reflect this, with those no longer required being removed.
  3. Regular, but not necessarily frequent, reviews of who holds which roles to confirm the procedures are working and that the contractor who left six months ago doesn't still have an active account!
  4. Limiting the number accessible in a day to a reasonable maximum, access location restrictions and where feasible, time of day restrictions.

55

55

Thank you for your attention!

56

56